UNIVERSITY OF ENGINEERING AND TECHNOLOGY

PESHAWAR

Department of Electrical Engineering

LABORATORY REPORT

Subject:	EE-170L Computer Programming Lab
Lab No:	Lab 09
Lab Title:	Introduction to Functions in C++
Semester:	Semester 2
Session:	Spring 2026

Submitted By:

Name: Muhammad Adil

Registration No: BF25NWELE0669

1. OBJECTIVE

To learn and apply functions in C++ including function declaration, prototyping, calling, and parameter passing. To understand how functions enable code reusability and modular programming.

2. THEORY

A function is a named section of code that performs a specific task. Functions allow code reusability and modular programming. A function has a name identifier, accepts parameters, and returns a result.

Key Concepts: (1) Function Declaration/Prototype: Tells the compiler about the function name, return type, and parameters. Syntax: returnType functionName(parameterList); (2) Function Definition: Contains the actual body of the function. (3) Function Call: Invokes the function to execute its code. (4) Parameters: Variables that receive values passed to the function. (5) Return Value: The result that a function sends back to the calling program.

Functions can be: without parameters and without return value (void func()), with parameters but without return value (void func(int a)), and with parameters and with return value (int add(int a, int b)).

3. LAB TASKS

3.1 Function Introduction and Declaration (greet function)

Declare a function called 'greet' that takes no parameters and returns nothing. Inside the function, display a greeting message of your choice. Call the function from the main program to execute it.

Source Code:

```
#include <iostream>
using namespace std;

// ============================================
// Function Prototype Declaration
// ============================================
// greet() - A function with no parameters and no return value
// This function displays a greeting message when called
void greet();

int main()
{
    // ============================================
    // Main Program
    // ============================================
    cout << "Program started..." << endl;

    // ============================================
    // Function Call
```

```
    // ============================================
    // Calling the greet function to execute its code
    greet();  // Function call - transfers control to greet()

    cout << "Back in main function." << endl;

    return 0;  // Successful program termination
}

// ============================================
// Function Definition
// ============================================
// greet() function definition
// This function takes no parameters and returns nothing (void)
void greet()
{
    // Display a personalized greeting message
    cout << "========================================" << endl;
    cout << "  Welcome to C++ Programming Lab!     " << endl;
    cout << "  Hello, Muhammad Adil!                 " << endl;
    cout << "  Have a great learning experience!     " << endl;
    cout << "========================================" << endl;
}
```

Expected Output:

```
Program started...
========================================
  Welcome to C++ Programming Lab!
  Hello, Muhammad Adil!
  Have a great learning experience!
========================================
Back in main function.
```

3.2 Basic Function Declaration and Call (sayHello function)

Declare a function called 'sayHello' that takes no parameters and returns nothing. Inside the function, display the message 'Hello, World!'. Call the function from the main program to execute it.

Source Code:

```
#include <iostream>
using namespace std;

// ============================================
// Function Prototype
// ============================================
// sayHello() - Function with no parameters, no return value
```

```
// Displays the classic 'Hello, World!' message
void sayHello();

int main()
{
    // ============================================
    // Main Program
    // ============================================
    cout << "Program started..." << endl;

    // Call sayHello function to display the message
    sayHello();

    cout << "Program ended successfully." << endl;

    return 0;
}

// ============================================
// Function Definition
// ============================================
// sayHello() - Displays 'Hello, World!' message
void sayHello()
{
    cout << "Hello, World!" << endl;  // Display the greeting message
}
```

Expected Output:

```
Program started...
Hello, World!
Program ended successfully.
```

3.3 Array Sum and Average using Function

Write a C++ program that takes 10 integer values as input from the user and stores them in a 1D array. Creates a function named calculateSum that accepts the array and its size as parameters and returns the sum of all elements. In the main() function, call the function, calculate and display the average of the array elements.

Source Code:

```
#include <iostream>
using namespace std;

// ============================================
// Function Prototype
// ============================================
// calculateSum() - Calculates sum of array elements
```

```cpp
// Parameters: arr[] - array of integers
//             size  - number of elements in array
// Returns: int - sum of all elements
int calculateSum(int arr[], int size);

int main()
{
    // ============================================
    // Variable Declaration
    // ============================================
    const int SIZE = 10;           // Constant for array size
    int numbers[SIZE];             // Array to store 10 integers
    int sum;                       // Variable to store sum
    float average;                 // Variable to store average

    // ============================================
    // Input Section
    // ============================================
    cout << "Enter " << SIZE << " integer values:" << endl;
    cout << "--------------------------------" << endl;

    // Loop to take input from user
    for (int i = 0; i < SIZE; i++)
    {
        cout << "Enter value " << (i + 1) << ": ";
        cin >> numbers[i];  // Store input in array at index i
    }

    // ============================================
    // Function Call to Calculate Sum
    // ============================================
    // Call calculateSum function passing array and its size
    sum = calculateSum(numbers, SIZE);

    // ============================================
    // Calculate Average
    // ============================================
    // Average = Sum / Number of Elements
    average = static_cast<float>(sum) / SIZE;

    // ============================================
    // Display Results
    // ============================================
    cout << "\n========== RESULTS ==========" << endl;
    cout << "Sum of all elements: " << sum << endl;
    cout << "Average of elements: " << average << endl;
    cout << "===============================" << endl;

    return 0;
```

```
}

// ============================================
// Function Definition: calculateSum
// ============================================
// This function calculates the sum of all elements in an array
// Parameters:
//   arr[] - the array containing integer values
//   size  - the number of elements in the array
// Returns:
//   The sum of all elements (integer)
int calculateSum(int arr[], int size)
{
    int total = 0;  // Initialize sum to 0

    // Loop through each element and add to total
    for (int i = 0; i < size; i++)
    {
        total += arr[i];  // Add current element to total sum
    }

    return total;  // Return the calculated sum
}
```

Expected Output:

```
Enter 10 integer values:
--------------------------------
Enter value 1: 10
Enter value 2: 20
Enter value 3: 30
Enter value 4: 40
Enter value 5: 50
Enter value 6: 60
Enter value 7: 70
Enter value 8: 80
Enter value 9: 90
Enter value 10: 100

========== RESULTS ==========
Sum of all elements: 550
Average of elements: 55
===============================
```

4. CONCLUSION

This lab introduced the fundamental concepts of functions in C++. We learned how to declare function prototypes, define functions, and call them from the main program. The tasks demonstrated functions without parameters (greet, sayHello), and functions with parameters and return values (calculateSum). Using functions for array operations like calculating sum makes code modular, reusable, and easier to maintain. Functions are essential building blocks for structured and object-oriented programming.